Triggers et procédures stockées

Document étudiant

Gestion du stock d’une boutique informatique

Niveau : BTS SIO • Durée : 3 à 4 heures
Environnement : MySQL 8.0 / 8.4 et MySQL Workbench

Objectifs

Créer et tester des triggers ; exploiter OLD et NEW ; écrire des procédures avec IN et OUT ; utiliser variables, conditions et erreurs métier.

Prérequis

Savoir écrire SELECT, INSERT et UPDATE et comprendre les clés primaires et étrangères.

Organisation

Exécuter les exercices dans l’ordre, sur la même base. Les résultats attendus supposent que chaque opération réussie n’est exécutée qu’une fois. Exécuter séparément les tests qui doivent échouer.

Le script d’initialisation efface uniquement la base tp_stock : l’utiliser dans un environnement de TP.

Livrables

Un script SQL commenté, les tests avec résultats observés et les réponses aux questions.

Nom : ..... Classe : .....

Repères avant de commencer

Trigger et procédure

Un trigger s'exécute automatiquement pour chaque ligne concernée par INSERT, UPDATE ou DELETE. BEFORE intervient avant l'opération ; AFTER intervient après. Une procédure est appelée explicitement avec CALL et accepte des paramètres.

OLD et NEW

INSERT : NEW seulement. UPDATE : OLD et NEW. DELETE : OLD seulement. OLD.prix est la valeur précédente ; NEW.prix est la nouvelle valeur.

Délimiteur et erreur métier

```
DELIMITER $$
CREATE PROCEDURE exemple()
BEGIN
  SELECT 'Bonjour';
END$$
DELIMITER ;
CALL exemple();

-- Dans un bloc BEGIN ... END :
IF condition THEN
  SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Message explicatif';
END IF;
```

DELIMITER est une commande du client, qui permet de transmettre un bloc contenant des points-virgules au serveur. Les variables DECLARE sont placées au début du bloc.

Paramètres et variables

IN : valeur reçue. OUT : valeur retournée à l'appelant. Une variable @nom appartient à la session ; une variable DECLARE appartient à son bloc.

Installation de la base

```
-- Attention : reinitialisation de la base de TP.
DROP DATABASE IF EXISTS tp_stock;
CREATE DATABASE tp_stock
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE tp_stock;

CREATE TABLE produit (
  id_produit INT AUTO_INCREMENT PRIMARY KEY,
  designation VARCHAR(100) NOT NULL,
  prix DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0
) ENGINE = InnoDB;

CREATE TABLE historique_prix (
  id_historique INT AUTO_INCREMENT PRIMARY KEY,
  id_produit INT NOT NULL,
  ancien_prix DECIMAL(10,2) NOT NULL,
  nouveau_prix DECIMAL(10,2) NOT NULL,
  date_modification DATETIME NOT NULL
    DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_historique_produit
    FOREIGN KEY (id_produit)
    REFERENCES produit(id_produit)
) ENGINE = InnoDB;

INSERT INTO produit (designation, prix, stock) VALUES
  ('Clavier USB',      25.00, 20),
  ('Souris sans fil',  18.50, 35),
  ('Ecran 24 pouces', 149.90, 10),
  ('Casque audio',     45.00, 15),
  ('Webcam HD',        39.90,  0);

SELECT * FROM produit;
SELECT * FROM historique_prix;
```

Questions : identifier les clés ; expliquer la clé étrangère ; justifier DECIMAL pour un prix ; distinguer zéro et NULL.

Exercice 1 / Contrôler l’ajout d’un produit

Notions : Trigger BEFORE INSERT

Travail demandé

Créer trg_produit_bi_controle sur produit. Refuser un prix inférieur ou égal à zéro et un stock négatif. Messages : « Le prix doit etre strictement positif » et « Le stock ne peut pas etre negatif ».

```
DELIMITER $$
CREATE TRIGGER trg_produit_bi_controle
BEFORE INSERT ON produit
FOR EACH ROW
BEGIN
  -- A completer
END$$
DELIMITER ;
```

Tests à exécuter séparément

```
INSERT INTO produit (designation, prix, stock)
VALUES ('Cle USB 64 Go', 12.90, 30); -- Reussite

INSERT INTO produit (designation, prix, stock)
VALUES ('Produit incorrect A', -5.00, 10); -- Erreur

INSERT INTO produit (designation, prix, stock)
VALUES ('Produit incorrect B', 15.00, -3); -- Erreur

INSERT INTO produit (designation, prix, stock)
VALUES ('Produit gratuit', 0, 5); -- Erreur

SELECT * FROM produit;
```

Questions et observations

Les produits refusés sont-ils présents ? Quel événement déclenche le contrôle ?

Réponse / observation :

.....

.....

Exercice 2 / Contrôler les modifications

Notions : Trigger BEFORE UPDATE

Travail demandé

Créer trg_produit_bu_controle et appliquer les mêmes règles lors des modifications.

Tests à exécuter séparément

```
UPDATE produit SET prix = 27.00
WHERE id_produit = 1; -- Reussite

UPDATE produit SET stock = -1
WHERE id_produit = 1; -- Erreur

UPDATE produit SET prix = 0
WHERE id_produit = 1; -- Erreur

SELECT * FROM produit WHERE id_produit = 1;
```

Questions et observations

Pourquoi le trigger de l’exercice 1 ne suffit-il pas ? À la fin, le prix doit être 27.00 et le stock 20.

Réponse / observation :

.....

.....

Exercice 3 / Historiser les changements de prix

Notions : Trigger AFTER UPDATE • OLD et NEW

Travail demandé

Créer trg_produit_au_historique. Si le prix change, insérer l'identifiant du produit, l'ancien prix et le nouveau prix dans historique_prix. Ne rien ajouter si seul le stock change ou si le prix reste identique.

Tests à exécuter séparément

```
UPDATE produit SET prix = 29.90
WHERE id_produit = 1;
SELECT * FROM historique_prix;

UPDATE produit SET stock = stock + 5
WHERE id_produit = 1;
SELECT * FROM historique_prix;

UPDATE produit SET prix = prix
WHERE id_produit = 1;
SELECT * FROM historique_prix;
```

Questions et observations

Une seule ligne doit indiquer le passage de 27.00 à 29.90. Pourquoi comparer OLD.prix et NEW.prix ? Pourquoi AFTER ? Que se passe-t-il si plusieurs lignes sont modifiées ?

Réponse / observation :

.....

.....

Exercice 4 / Consulter les stocks faibles

Notions : Procédure avec paramètre IN

Travail demandé

Créer sp_produits_stock_faible(IN p_seuil INT). Refuser un seuil NULL ou négatif. Afficher les produits dont le stock est inférieur ou égal au seuil, triés par stock puis par identifiant.

Tests à exécuter séparément

```
CALL sp_produits_stock_faible(10);
CALL sp_produits_stock_faible(0);
CALL sp_produits_stock_faible(-1);   -- Erreur
CALL sp_produits_stock_faible(NULL); -- Erreur
```

Questions et observations

Quelle différence existe-t-il entre le paramètre p_seuil et la colonne stock ? Quel produit possède initialement un stock nul ?

Réponse / observation :

.....

.....

Exercice 5 / Réapprovisionner un produit

Notions : Variables locales • SELECT INTO • IF

Travail demandé

Créer sp_reapprovisionner(IN p_id_produit INT, IN p_quantite INT). Refuser une quantité NULL, nulle ou négative. Vérifier que le produit existe, augmenter son stock et afficher son identifiant, sa désignation et son stock.

```
DECLARE v_existe INT DEFAULT 0;
SELECT COUNT(*) INTO v_existe
FROM produit WHERE id_produit = p_id_produit;
```

Tests à exécuter séparément

```
CALL sp_reapprovisionner(5, 8);    -- Stock = 8
CALL sp_reapprovisionner(999, 5);  -- Erreur
CALL sp_reapprovisionner(1, -2);   -- Erreur
CALL sp_reapprovisionner(1, NULL); -- Erreur
SELECT * FROM produit WHERE id_produit = 5;
```

Questions et observations

Comment vérifier l’existence du produit ? Pourquoi tester aussi IS NULL ? Les appels refusés ont-ils changé les stocks ?

Réponse / observation :

.....

.....

Exercice 6 / Calculer la valeur d’un stock

Notions : Procédure avec paramètre OUT

Travail demandé

Créer sp_valeur_stock(IN p_id_produit INT, OUT p_valeur DECIMAL(12,2)). Vérifier que le produit existe et placer prix × stock dans le paramètre de sortie.

Tests à exécuter séparément

```
CALL sp_valeur_stock(5, @valeur);
SELECT @valeur AS valeur_stock; -- 319.20

CALL sp_valeur_stock(999, @valeur); -- Erreur
```

Questions et observations

Quel est le rôle de OUT ? Pourquoi utiliser @valeur ? Une variable locale déclarée avec DECLARE est-elle accessible après CALL ?

Réponse / observation :

.....

.....

Exercice 7 / Vendre un produit

Notions : Procédure et triggers combinés

Travail demandé

Créer sp_vendre(IN p_id_produit INT, IN p_quantite INT). Refuser une quantité NULL, nulle ou négative et un produit inexistant. Diminuer le stock seulement s'il est suffisant, sinon signaler une erreur. Afficher le stock restant.

Vérifier la disponibilité et diminuer le stock dans une même instruction, puis lire immédiatement ROW_COUNT().

```
UPDATE produit SET stock = stock - p_quantite
WHERE id_produit = p_id_produit
  AND stock >= p_quantite;
```

Tests à exécuter séparément

```
CALL sp_vendre(5, 3);   -- Stock = 5
CALL sp_vendre(5, 50);  -- Erreur, stock reste a 5
CALL sp_vendre(999, 1); -- Erreur
CALL sp_vendre(5, 0);   -- Erreur
SELECT * FROM produit WHERE id_produit = 5;
```

Questions et observations

Quels triggers sont exécutés ? Pourquoi aucune ligne de prix n'est-elle ajoutée ? Quel risque présente une lecture du stock suivie d'une modification indépendante ?

Réponse / observation :

.....

.....

Bilan et travail à rendre

Inspecter les objets

```
SHOW TRIGGERS FROM tp_stock;
SHOW PROCEDURE STATUS WHERE Db = 'tp_stock';
SHOW CREATE TRIGGER trg_produit_bi_controle;
SHOW CREATE PROCEDURE sp_vendre;

-- Pour corriger puis recreer un objet :
DROP TRIGGER IF EXISTS trg_produit_bi_controle;
DROP PROCEDURE IF EXISTS sp_vendre;
```

Compte rendu des tests

Pour chaque test : noter l'état avant, l'instruction, le résultat attendu et le résultat observé. Vérifier aussi les données après une erreur.

Checklist de remise

1. Les trois triggers et les quatre procédures.
2. Les tests valides et invalides.
3. Les réponses aux questions.
4. Une explication du rôle de chaque objet.

Pour aller plus loin

Créer un historique des stocks ; calculer la valeur totale avec SUM ; appliquer une hausse à tous les prix ; tester une modification et son historique dans une transaction suivie de ROLLBACK.

Documentation

Manuel MySQL : dev.mysql.com/doc/refman/8.0/en/
Rubriques : Trigger Syntax and Examples ; CREATE PROCEDURE ; SIGNAL ; Defining Stored Programs.